

GNN–BERT Fusion for Multi-Label Music Context Understanding

CSE425 Neural Networks Project

Shahrin Farah Tasnim

Department of Computer Science and Engineering

BRAC University

Dhaka, Bangladesh

shahrin.farah.tasnim@g.bracu.ac.bd, sftmaria12345@gmail.com

Abstract—Musical context spans genre, mood, harmonic structure, and timbral texture, and these attributes are expressed jointly rather than independently. Sequence models applied to spectrograms capture local acoustic patterns but do not represent relational structure such as repeated segments or chord transitions. We implement a hybrid system combining a Graph Neural Network over music structure graphs with a BERT text encoder, fused through cross-attention, and evaluate it on two datasets. On GTZAN, a GraphSAGE encoder over segment-similarity graphs reaches 0.6362 ± 0.0163 macro-F1 across three seeds for ten-way genre classification, substantially outperforming a GAT encoder under matched hyperparameters. On paired MusicCaps audio and captions, cross-attention fusion attains 0.4189 AUC-PR, ahead of early concatenation (0.4034), text alone (0.3643), and audio graphs alone (0.1692), confirming that both modalities contribute and that attention-based fusion outperforms naive concatenation. We also report a preprocessing finding: chord template matching at the STFT frame rate produces chord-transition graphs dominated by self-loops, and pooling chroma to chord-length blocks before matching removes this degeneracy entirely. All reported numbers come from executed runs; components that were implemented but not evaluated are identified as such.

Index Terms—graph neural networks, music information retrieval, multi-modal fusion, BERT, cross-attention, multi-label classification

I. INTRODUCTION

A single music clip simultaneously carries genre and era, mood and emotional arousal, harmonic structure, lyrical semantics, and production style. Models operating purely on spectrogram sequences learn local acoustic patterns well, but represent a track as an undifferentiated stream. They have no explicit mechanism for expressing that a chorus recurs three times, that a chord progression returns to its tonic, or that a textual description corresponds to a specific structural passage.

This project builds a system with two complementary encoders. A Graph Neural Network operates over graphs whose nodes are time segments or chords and whose edges encode temporal adjacency, acoustic similarity, or observed chord transitions. A BERT encoder operates over natural-language captions and tag vocabularies. A cross-attention layer fuses the two representations, and the fused vector drives multi-label prediction over music context tags. Our contributions are as follows:

- A complete preprocessing pipeline mapping raw audio to graph structures, with per-track feature normalization and cross-family scale balancing (Section III).
- An empirical finding that frame-rate chord estimation degenerates the chord-transition graph, together with a pooling-and-collapsing fix that resolves it (Section III-C).
- Genre classification results on GTZAN comparing GraphSAGE and GAT encoders across three random seeds (Section IV-A).
- A four-way fusion ablation on paired MusicCaps audio and captions, isolating the contribution of each modality and of the attention mechanism (Section IV-B).

II. PROBLEM FORMULATION AND DATA

A. Problem Definition

A music track is represented as a tuple $T = (X_{\text{audio}}, X_{\text{text}}, G, y)$, where X_{audio} is a log-mel or chroma representation over T frames, X_{text} is a tokenized caption or tag string, $G = (V, E)$ is a music structure graph, and y is a multi-label context vector.

The BERT encoder maps text to contextual embeddings $H_{\text{text}} = \text{BERT}(X_{\text{text}}) \in \mathbb{R}^{L \times d}$. The GNN encoder updates node features through L rounds of message passing,

$$h_i^{(l+1)} = \sigma \left(W^{(l)} \cdot \text{AGG} \left(h_i^{(l)}, \{h_j^{(l)} : j \in \mathcal{N}(i)\} \right) \right), \quad (1)$$

where $h_i^{(0)}$ is initialized from segment audio features. A fusion readout combines the graph-level representation g with the text representation, and a sigmoid output layer produces per-tag probabilities. Training minimizes binary cross-entropy across K tags,

$$\mathcal{L} = - \sum_{k=1}^K [y_k \log \hat{y}_k + (1 - y_k) \log(1 - \hat{y}_k)]. \quad (2)$$

B. Datasets

GTZAN [1] provides 1,000 thirty-second clips across ten genres at 22,050 Hz. One file, `jazz.00054`, is corrupt in

the original release and is excluded, leaving 999 usable tracks. GTZAN is used for the graph-only genre classification task.

MusicCaps [7] provides 5,521 clips with expert-written natural-language captions and accompanying lists of short aspect phrases. The official release distributes YouTube identifiers rather than audio. We therefore use the CLAPv2/MusicCaps community mirror, which pairs audio with captions directly and contains 5,352 rows; the shortfall relative to the official release consists of AudioSet clips no longer retrievable. This mirror is what makes the fusion experiment feasible, since it requires audio and text for the same clip.

III. PREPROCESSING

A. Feature Extraction and Normalization

Audio is resampled to 22,050 Hz and converted to mono. Three feature families are computed over the full track using `librosa` [8]: a 128-bin log-mel spectrogram, 12-bin chroma, and 20 MFCC coefficients, all with a 2048-sample FFT and 512-sample hop.

Two normalization decisions materially affect downstream performance.

Normalization is applied once per track, not per segment. An earlier implementation normalized each segment independently, forcing every segment to zero mean and unit variance and thereby erasing the loudness and timbre differences *between* segments. Those differences are precisely the signal a segment-similarity graph is intended to capture. Computing features over the full track and normalizing once preserves them.

Each feature family is standardized separately. Raw MFCC coefficient 0 routinely takes values in the hundreds, while normalized log-mel values lie near ± 1 . Concatenated without separate standardization, the MFCC dimensions dominate the node vector numerically and therefore dominate message passing regardless of their informativeness. After the correction, measured per-block standard deviations are 0.702 (mel), 0.707 (chroma), and 0.705 (MFCC), confirming the three families contribute at comparable scale.

B. Segmentation and Graph Construction

Tracks are divided into 5-second windows with a 2.5-second hop, giving 50% overlap. Each segment is summarized by the mean and standard deviation of each feature family over its frames, producing node vectors in $\mathbb{R}^{320}$ ($2 \times 128 + 2 \times 12 + 2 \times 20$). On a 30-second GTZAN clip this yields 12 nodes.

Segment-similarity graphs connect temporally adjacent segments and additionally connect any pair whose cosine similarity exceeds $\tau = 0.7$, allowing repeated structural material to link across time. On GTZAN this produces approximately 141 edges per graph.

A beat-synchronous segmentation path is also implemented using `librosa.beat.beat_track`. It emits mean and standard deviation per beat in the same block order as the windowed path, so both modes produce 320-dimensional nodes

and remain interchangeable without altering the model’s input dimension.

C. Chord Graphs and a Frame-Rate Failure Mode

Chord estimation matches chroma against 24 binary pitch-class templates covering 12 major and 12 minor triads, each L2-normalized.

Our initial implementation performed this matching at the native STFT frame rate. A 512-sample hop at 22,050 Hz corresponds to roughly 23 milliseconds per frame, whereas musical chords typically persist for one to two seconds. The consequence is severe: on a 30-second clip, frame-level matching produced 1,291 transitions, of which 71.4% were self-loops, and identified 22 distinct chords in a synthetic signal containing only four. The resulting graph is close to complete, and its edge weights encode frame duration rather than harmonic movement.

The correction pools chroma into one-second blocks before template matching, then collapses consecutive duplicates so that a sequence C C C G G becomes C G. Table I quantifies the effect on a synthetic C–G–Am–F progression.

TABLE I
EFFECT OF CHROMA POOLING AND DUPLICATE COLLAPSING ON CHORD-GRAPH STRUCTURE, MEASURED ON A SYNTHETIC FOUR-CHORD PROGRESSION

Method	Transitions	Self-loops	Unique chords
Frame-level matching	1,291	71.4%	22
Pooled to 1 s blocks	30	50.0%	4
Pooled and collapsed	15	0.0%	4

The corrected method recovers the intended progression exactly. Applied to a real GTZAN jazz clip, it produces 20 transitions across 10 distinct chords with zero self-loops. We additionally switched from `chroma_stft` to `chroma_cqt`, since constant-Q bins are log-spaced and aligned to musical pitch, making them more appropriate for harmonic analysis.

IV. EXPERIMENTS

A. Graph-Only Genre Classification

1) *Setup*: The 999 GTZAN tracks are split 70/15/15 by stratified sampling within each genre, giving 699 training, 150 validation, and 150 test graphs with identical genre proportions in each split. Labels are ten-dimensional one-hot vectors. Both encoders, GraphSAGE [3] and GAT [4], use two layers, hidden dimension 128, and dropout 0.3, implemented with PyTorch Geometric [9]. Training runs for 30 epochs with AdamW at learning rate 2×10^{-4} and weight decay 10^{-5} . Graph-level representations are obtained by mean pooling over node embeddings. We run three seeds (42, 123, 2024) per architecture.

2) *Results*: Table II reports validation performance.

For reference, random assignment over ten balanced classes yields approximately 0.10 macro-F1, so the graph representation carries substantial genre information despite using no spectrogram CNN and no text.

TABLE II
GENRE CLASSIFICATION ON GTZAN. GRAPH SAGE RESULTS ARE MEAN AND STANDARD DEVIATION OVER THREE SEEDS

Encoder	Macro-F1	Micro-F1	AUC-PR
GraphSAGE	0.6362 ± 0.0163	0.6319	0.7288
GAT	0.3237	0.4333	0.5547

3) *Analysis*: GraphSAGE outperforms GAT by a wide margin, and the training curves indicate why. GraphSAGE training loss falls from 0.4164 to 0.1829 over 30 epochs, while GAT plateaus at 0.2644 after descending from 0.4229. GAT is therefore underfitting rather than overfitting.

We attribute this to graph topology. With 12 nodes and roughly 141 edges, each node is adjacent to approximately 12 neighbours, so the graphs are close to complete. Attention must learn which neighbours matter, but in a near-complete graph over 12 nodes there is little neighbourhood structure to discriminate between, and the additional attention parameters cost more in optimization difficulty than they recover in representational power. GraphSAGE’s mean aggregation is better matched to this regime.

This comparison holds hyperparameters constant across encoders. GAT was not separately tuned, in particular its attention head count and learning rate. The result therefore shows GAT underperforming under matched settings, not that GAT is inferior in general.

B. Text Tagging and GNN–BERT Fusion

1) *Setup*: From the paired MusicCaps mirror we processed 800 clips with zero decode failures. Restricting to clips carrying at least one of the 20 most frequent aspects leaves 667 clips, split 467/100/100. The most frequent aspects are *low quality*, *instrumental*, *emotional*, *passionate*, *noisy*, *amateur recording*, *medium tempo*, and *energetic*.

Graphs are built from clip audio using the pipeline of Section III. Captions are encoded with DistilBERT [6], a distilled variant of BERT [5]. Cross-attention fusion follows

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right), \quad Q = gW_Q, \quad K = H_{\text{text}}W_K, \quad (3)$$

$$z = \text{CONCAT}(g, AH_{\text{text}}), \quad (4)$$

with graph dimension 128 and text dimension 768, giving $z \in \mathbb{R}^{896}$. The BERT encoder is frozen while the GNN, attention, and classification layers train, for two epochs at learning rate 10^{-3} with batch size 16. Freezing BERT was a compute-budget decision, discussed in Section VI.

2) *Ablations*: We evaluate four configurations. *GNN only* uses the audio graph alone. *BERT only* uses the caption alone. *Early concatenation* joins the graph readout with the text CLS token without attention. *Cross-attention* is the full fusion model. All four share identical data, splits, and training budget, so differences are attributable to the fusion mechanism rather than to experimental setup.

TABLE III
FUSION ABLATION ON PAIRED MUSICCAPS AUDIO AND CAPTIONS, PREDICTING TOP-20 ASPECT LABELS. ALL VARIANTS SHARE IDENTICAL SPLITS AND TRAINING BUDGET

Model	Macro-F1	Micro-F1	AUC-PR
GNN only (audio graph)	0.0000	0.0000	0.1692
BERT only (text)	0.0276	0.0737	0.3643
Early concatenation	0.0167	0.0377	0.4034
Cross-attention	0.0731	0.1810	0.4189

3) *Analysis*: Cross-attention fusion leads on every metric. The ordering supports two claims. First, fusion outperforms either modality in isolation, indicating that the audio graph and the caption carry complementary information. Second, attention-based fusion outperforms naive concatenation by 0.0155 AUC-PR, indicating that allowing the graph representation to query specific caption tokens is more effective than appending a single pooled text vector.

We report AUC-PR as the primary comparison because it is threshold-free. With 467 training clips, 20 imbalanced tags, and a two-epoch budget, predicted probabilities rarely exceed the 0.5 decision threshold, which suppresses F1 uniformly across variants. The GNN-only row illustrates this directly: it scores 0.0000 macro-F1 because it never crosses the threshold, yet its AUC-PR of 0.1692 shows it ranks tags better than chance. Reporting F1 alone would incorrectly suggest the audio branch learned nothing.

Absolute values across this table are low. They reflect the small training set and short schedule rather than a ceiling on the method; the cross-attention model was still improving when training stopped, with AUC-PR rising from 0.3479 after one epoch to 0.4189 after two.

V. IMPLEMENTED BUT NOT EVALUATED

Two components are present in the codebase but produced no results, and we report no numbers for them.

Contrastive retrieval. A dual-encoder with the InfoNCE objective and Recall@K retrieval metrics is implemented in `src/contrastive.py`. The project specification designates this component as optional. It was not run.

CNN mel-spectrogram baseline. `src/cnn_baseline.py` implements a four-block convolutional network operating directly on log-mel spectrograms, using the same stratified split as the GNN so the comparison would be fair. Feature extraction completed for all 699 training tracks, but the training run did not finish within the available compute budget. The GraphSAGE-versus-GAT comparison in Section IV-A therefore serves as our architecture-level baseline, and the GNN-only row of Table III serves as the audio-only reference for the fusion experiments.

VI. LIMITATIONS

GTZAN label integrity. GTZAN is known to contain replicated excerpts, mislabeled tracks, and distorted recordings [2], and measured agreement between expert annotators and its ground-truth labels is approximately 59%. Because repeated

excerpts may appear in both training and test partitions under a random split, our figures should be read as in-distribution performance on a dataset with known label noise, not as a clean estimate of genre classification accuracy.

Caption-label overlap in MusicCaps. Captions and aspect labels are written by the same annotator for the same clip, so aspect phrases frequently appear verbatim within captions. This makes part of the text-based task closer to extraction than inference and inflates the BERT-only and fusion rows relative to predicting tags from audio. Our implementation provides a masking option that strips exact aspect substrings from captions so this effect can be measured rather than assumed; the ablation was not run for lack of time. The GNN-only row is unaffected and is the cleanest audio-only reference in Table III.

Training budget. The fusion experiments ran for two epochs on CPU with a frozen text encoder. Fine-tuning BERT and training longer would very likely raise all rows of Table III; the relative ordering is our claim, not the absolute values.

Single-seed fusion results. Table III reports one seed per configuration. The gap between cross-attention and GNN-only is large, but the 0.0155 AUC-PR margin over early concatenation lies within the range seed variance could plausibly explain, and we do not claim significance without replication.

Graph size. Thirty-second clips at 5-second windows yield only 12 nodes. This limits how much relational structure message passing can exploit and is a plausible contributor to GAT’s underperformance. Longer recordings or finer segmentation would produce richer graphs.

VII. CONCLUSION

We implemented a GNN-BERT system for multi-label music context understanding and evaluated it on two datasets. Graph structure alone supports genre classification on GTZAN at 0.6362 ± 0.0163 macro-F1 with a GraphSAGE encoder, well above chance and above the GAT alternative under matched hyperparameters. On paired MusicCaps data, cross-attention fusion outperforms both single-modality baselines and naive concatenation, supporting the hypothesis that structural and semantic representations are complementary.

We also documented a preprocessing failure mode relevant to anyone building chord-transition graphs: matching chord templates at the STFT frame rate yields graphs whose edge weights encode frame duration rather than harmonic movement, with over 70% of transitions being self-loops. Pooling chroma to chord-length blocks and collapsing repeats removes the degeneracy entirely.

The clearest next steps are running the masked-caption ablation to quantify lexical overlap, completing the CNN baseline, replicating the fusion ablation across seeds, and evaluating the contrastive retrieval component.

REPRODUCIBILITY

Source code, per-run metrics, and configuration are available at <https://github.com/SHAHNIN-Farah-Tasnim/GNN-Bert-Music-Generation>. Every run writes a JSON record containing its seed, hyperparameters, split

sizes, and metrics. The experiments reported here are reproduced by running `src/preprocess_gtzan.py`, then `src/train.py` with `--task gnn_tag`, then `src/fusion_musiccaps.py` once per fusion mode.

REFERENCES

- [1] G. Tzanetakis and P. Cook, “Musical genre classification of audio signals,” *IEEE Trans. Speech Audio Process.*, vol. 10, no. 5, pp. 293–302, 2002.
- [2] B. L. Sturm, “An analysis of the GTZAN music genre dataset,” in *Proc. 2nd ACM Workshop on Music Information Retrieval with User-Centered and Multimodal Strategies*, 2012, pp. 7–12.
- [3] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems*, 2017, pp. 1024–1034.
- [4] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. Int. Conf. Learning Representations*, 2018.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [6] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [7] A. Agostinelli *et al.*, “MusicLM: Generating music from text,” *arXiv preprint arXiv:2301.11325*, 2023.
- [8] B. McFee *et al.*, “librosa: Audio and music signal analysis in Python,” in *Proc. 14th Python in Science Conf.*, 2015, pp. 18–25.
- [9] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.